Title: TAS-042: Model inference in SQL
Status: Discussion Paper
Date: Sept 26, 2026
Authors: Bilge Ince , Peter Eisentraut

Abstract

This paper discusses and provides examples of what kind of model inference functions could be added. It proposes mainly three things: the structure of a function library document for model inference in SQL, a fixed entry template that every function in that library is described with, and a short set of cross-cutting concepts that have to be settled before any individual function can be specified at all.

References

[W32-023] "AI functions"; Jan-Eike Michels; July 17, 2026

1. Introduction

1.1 Scope

This document outlines the specification of a library of functions whose evaluation involves invoking a machine learning model. It covers text input (binary: see §2.6), generation, embedding, ranking, and aggregation over groups. This function library could be an addition to the 29075 series of standards.

Out of scope: how a model is deployed, trained, hosted, billed, or authenticated; model selection; prompt content; agent or tool-calling protocols; and performance, consistent with [W32-023] §1.5.

1.2 Relationship to SQL/Foundation

Three items in this document are language-level rather than library-level and are suggested for SQL/Foundation (or GQL) instead, because each needs a change to the language itself and because the VECTOR type they attach to is already in the CD:

1. the model reference (§2.1);
2. embedding space comparability (§2.2);
3. the rank fusion construct (§8.1).

This paper lays out some requirements for these specifications but contains no specific syntax proposals.

1.3 Terminology and naming

This document uses *model*, not *LLM*: several entries invoke no language model (embedding, reranking, parsing), and the input is not always a prompt. *Inference* means one invocation of a model. *Model reference* is defined in §2.1.

The naming of the functions themselves is an open issue. Three options:

Option	**Example**	**For**	**Against**
a. AI_ prefix	AI_GENERATE	matches all surveyed implementations	names a market category, not an invariant; describes nothing checkable
b. bare verbs	GENERATE, EMBED	shortest	high collision risk with user identifiers
c. MODEL_ prefix or equivalent	MODEL_GENERATE	says what is actually invoked; stable across model fashions; low collision	diverges from shipped products

This document uses option c with lowercase names and treats the AI_ spellings as possible implementation-defined synonyms. The intent is to keep the discussion about what the functions guarantee rather than about what they are called.

1.4 Inclusion criteria

Note: This clause is the filter that keeps the library from becoming an arbitrary list. Proposed as policy for the group to adopt or reject.

A function is admitted to this library only if all of the following hold.

1. **Two implementations already exist:** At least two SQL systems ship the function, and they behave similarly enough that we can write a single entry for it. There is no need to split the description into "this vendor does X, that vendor does Y."
2. **Not derivable in one line:** You can't recreate it in one line by combining functions we already document. If something is just shorthand for, say, two MODEL_EMBED calls plus a distance comparison, we either label it as shorthand or leave it out.
3. **At least one testable invariant:** There has to be some behaviour an implementation can be checked against and one that holds no matter which model is used (see §2.7). If every part of what a function does is left up to the implementation and can't be tested, then it's just a name, not a real specification.

4. **Its name doesn't promise something no one can deliver:** If a name implies a guarantee about recall or safety that no implementation actually provides, we reject it, even though lots of systems ship it.

Functions that fail only the first test (not enough implementations yet) aren't thrown out. We record them as candidates, note why they didn't qualify, and look at them again later.

2. Cross-cutting concepts

2.1 Model reference

A model can be referable as a named schema object with declared properties, not only as a character string naming an endpoint. The declared properties are the minimum needed to type a function invocation and to decide whether two results can be compared:

Property	**Why the language needs it**
model kind: generation, embedding, ranking, classification	selects which functions accept the reference at all
input modality: character, binary (with media types), vector	see §2.6; an entry restricted to character input excludes the main document use cases
result type: character, VECTOR(n), numeric, row type	a scalar function whose result type is decided by a runtime string cannot be specified
output dimensionality and space identity, for embedding models	see §2.2
input capacity, in implementation-defined units	required by the aggregate and summarizing entries to decide whether chunking is needed
determinism controls supported	§2.3; determines which determinism class the implementation may claim

Everything else (transport, credentials, provider, hyperparameter defaults, cost) remains implementation-defined, following the registered-XML-schema precedent cited in [W32-023] §1.4.

2.2 Embedding space comparability

Two vectors are comparable only if produced by the same model, the same version, and the same pooling and normalization. Equal dimensionality is not comparability. Two 768-dimension encoders produce vectors in unrelated spaces; a distance operator applied across them returns a well-formed ranking that

means nothing, with no error and no warning. This is the most likely silent failure in the entire surface, and it is reachable today with the VECTOR type already in the 9075 CD, independently of whether this library is ever published.

Requirements:

1. A vector value's space of origin shall be recoverable, either by typing the vector on the space or by an implementation-defined space identifier associated with the column.
2. Comparing vectors from different spaces shall raise a defined condition (§2.4) rather than return a result.
3. The mechanism by which an implementation determines space identity is implementation-defined; the failure mode is not.

NOTE: Implementations already compare vectors with operators. These operators do not check that both vectors come from the same embedding space. Requirement 2 applies to operations like these, so this library does not define its own similarity function.

2.3 Determinism, volatility, and evaluation count

Every entry declares one determinism class:

Class	**Meaning**
DETERMINED	identical input, model reference, and configuration yield identical results within a session; the implementation guarantees it
REPRODUCIBLE ON REQUEST	identical results when the implementation's determinism controls (for example seed, temperature) are set; otherwise unconstrained
UNDETERMINED	no guarantee

Two characteristics are important when you define a function: Undetermined and have external action. That already constrains their use in generated columns, index expressions, constraint checks, and anywhere else determinism is required, and it constrains the rewrites an implementation may perform.

Two points to state explicitly in this clause, because applications will otherwise assume the opposite:

- The number of times an invocation is evaluated for a given row is implementation dependent. An implementation may evaluate a MODEL_FILTER predicate more than once for a row, or once for a group of identical inputs. Applications shall not depend on the count. Each evaluation may have monetary cost.
- Memoization of identical (model reference, input, configuration) invocations within a statement is permitted and not required.

2.4 Conditions and degraded results

Failure modes here do not map onto existing SQL conditions.
Proposed: one new SQLSTATE class with subconditions.

Condition	**Notes**
model not available	includes unreachable remote model
model not authorized	
resource limit exceeded	quota, rate limit, budget
input exceeds model capacity	the caller-visible form of the context window
result truncated	see below
result withheld by policy	refusal or content filter; distinguishable from an empty result
result does not conform to requested schema	§2.5
incomparable embedding spaces	§2.2

Truncation deserves separate treatment. A response cut off at a token limit is indistinguishable from a complete short answer, which makes it the failure most likely to reach a user as data. If the model runs out of room mid-sentence, the database should say so instead of handing you the fragment as if it were the answer.

Each entry states, for each condition, whether it raises an exception or returns the null value, and whether a per-invocation option selects between them.

2.5 Requested result schema

Where a function accepts a schema describing its result, two levels of support exist in practice and both should be recognized:

- **Level 1, validated:** the implementation validates the model output against the schema and raises the §2.4 condition on mismatch.
- **Level 2, constrained:** the implementation constrains decoding so that output cannot violate the schema.

Level 1 is implementable against any model. Level 2 depends on the specific model and the serving stack. To be compliant, an implementation must support Level 1 and may also support Level 2; the implementation must clearly state which levels it provides.

Do not write entries assuming Level 2 is available. If a result type comes from a schema argument, that argument must be known during the preparation stage. This follows the pattern used for the JSON_TABLE's column list in SQL/Foundation.

2.6 Input modality

Entries restricted to character input cover a minority of real uses. The common extraction case is a PDF or an image, not a text column. Each entry declares the modalities it accepts, and an entry that accepts binary input states how the media type is determined. Text-only entries are marked as such so that the gap is visible rather than implied.

2.7 Testable invariants and conformance

Nondeterministic output cannot be conformance-tested by comparing values. It does not follow that nothing can be tested. Each entry declares its invariants from this list:

1. **Signature and result type.** Result conforms to the declared or derived type.
2. **Null and condition propagation.** Null input, and each §2.4 condition, behave as the entry states.
3. **Declared determinism.** For a REPRODUCIBLE ON REQUEST claim, fixing the controls yields identical results across invocations.
4. **Schema conformance.** For level 1 or 2, output either conforms or raises.
5. **Score invariants.** For score-returning entries: declared range respected; identity (an input scored against itself is maximal); symmetry where claimed.
6. **Input size independence.** For aggregates: the function completes for groups whose total input exceeds the model's capacity. This is the one substantive semantic guarantee available for aggregate entries, and it is the reason they are worth standardizing at all.

An entry with no invariant from this list does not meet §1.4(3).

3. Function inventory

Note: This table is the working list.

Proposed name	Group	Status	Note
`model_generate`	5	specified, 5.1	
`model_embed`	6	specified, 6.1	absent from the W32-023 1.12 list; see 3.1
`similarity_cosine`	6	not admitted	the language provides it: comparison belongs to the VECTOR type (§1.2,

Proposed name	Group	Status	Note
			§2.2). Replaces AI_SIMILARITY from W32-023.
`summarize_collection`	7	recommended, 7.1	
`fuse_rankings`	8	proposed addition to SQL/Foundation	result defined in 8.1
`generate_boolean`	5	candidate	the predicate case; result typing per 4.2
`summarize_text`	5	candidate	shorthand for 5.1 under criterion 2
`extract_fields`	5	candidate	requires a result structure parameter
`rerank`	8	candidate	requires a ranking model kind
`model_classify`	5	specified 5.2	see below
`mask_text`	—	not admitted	criterion 4
`analyze_sentiment`	—	not admitted	criterion 2; `model_classify` with fixed labels

3.1 Missing embedding function

[W32-023] §1.12 lists eight functions and no embedding function, while listing AI_SIMILARITY, which cannot be defined without one, and while the VECTOR type is already in the CD with no standard way to populate it. Snowflake's AI_EMBED is in the survey at §1.8.1. This looks like an oversight rather than a decision, and it is worth correcting early: embedding is the one place in this surface where the result is durable data rather than a transient answer, which makes it both the most valuable entry and the one where getting the semantics wrong is most expensive.

4. Concepts

4.1 Introduction to model inference functions

The functions in this document fall into four groups:

- *generation functions*, whose result is produced by a generation model;
- *embedding functions*, whose result is a vector produced by an embedding model;
- *aggregation functions*, whose input is a collection and whose result is produced by repeated inference over parts of that collection;

- *ranking functions*, which operate on the results of retrieval and invoke a model.

4.2 Formalizing the input

4.2.1 Identification of the model

A function specified in this document that invokes a model takes a parameter of the record type model_descriptor. An invocation shall supply a value of this type.

```
SQL
RECORD model_descriptor (
  model_name        : CHARACTER STRING,
  model_version     : CHARACTER STRING,
  model_kind        : CHARACTER STRING, -- 'GENERATION', 'EMBEDDING', 'RANKING',
'CLASSIFICATION'
  input_capacity    : INTEGER,
  capacity_unit     : CHARACTER STRING,
  result_dimension  : INTEGER DEFAULT 0,
  space_identity    : CHARACTER STRING DEFAULT '',
  reproducible      : BOOLEAN DEFAULT False
)
```

model_descriptor lists the properties of a model reference (§2.1) that the functions in this library read. The model reference itself, and how it is created, are part of the proposed addition to SQL/Foundation (§1.2).

4.2.2 Settings of an Inference

A function specified in this document that invokes a model takes a parameter of the record type inference_settings.

```
SQL
RECORD inference_settings (
  max_result_length : INTEGER DEFAULT 0,
  randomness        : REAL    DEFAULT 0,
  seed              : INTEGER DEFAULT 0,
  extension         : CHARACTER STRING DEFAULT ''
)
```

max_result_length is the largest result the invocation is to produce, in the units given by capacity_unit of the model descriptor; 0 means no limit is imposed by the invocation. randomness shall be greater than or

equal to 0; the effect of a value greater than 0 is implementation-defined other than that a value of 0 shall be the least random setting the implementation offers. seed selects among the results obtainable at a given randomness; its effect is implementation-defined. The content of extension and its effect on an inference are implementation-defined.

NOTE: Extension exists so that implementation-specific settings do not require new parameters on every function. A conforming implementation may ignore it entirely.

4.3 Function definitions

```
None

N.n <Function name in sentence case>

    Purpose
        One sentence. What is returned, not how a model is used.

    Kind | Status

    Function Signature
        FUNCTION <name> (
          <parameter>  : <data type> [ DEFAULT <value> ]
        ) RETURNS <data type | record type>

    Parameter Definitions
        <parameter>: meaning, and the effect if no argument is provided.

    Result
        What the returned value means.

    Argument Considerations
        1) <restriction>. If <violated>, then an exception condition is
           raised: <condition name> (<code>).

    Formal Definition
        1) Let <X> be the value of the <parameter> parameter.
        ...
        n) The step at which a model is invoked is identified as
           implementation-defined (IDnnn).
        n) The result of the invocation of the function is <X>.

    Determinism
        Determined | Reproducible on request | Undetermined (2.3).
```

```
Comparability
    Whether results of two invocations may be compared, and under what
    condition (2.2). Omitted where results are not compared.

Conformance
    Without Feature LMnn, "<name>() function", conforming use of this
    document shall not invoke the <name> function.
```

5. Generation Functions

5.1 MODEL_GENERATE

Purpose

Returns the result of invoking a generation model on a supplied input.

Kind. Scalar. **Status.** Recommended. **Model kind.** Generation. **Input modality.** Character (§2.6).

Function Signature

```SQL
FUNCTION model_generate (
  input_text : CHARACTER STRING,
  model      : model_descriptor,
  settings   : inference_settings DEFAULT «default settings»
) RETURNS CHARACTER STRING
```

Parameter Definitions

`input_text:` the value given to the model.
`model:` the descriptor of the model to be invoked.
`settings:` the settings of the inference. If no argument is provided, implementation-defined default settings apply.

Result

Return the result of one inference of the model identified by model on input_text.

Argument Considerations

1. `model.model_kind` shall be `'GENERATION'`. If it is not, then an exception condition is raised: data exception as the invalid value for argument.
2. `settings.randomness` shall be greater than or equal to 0 (zero). If it is not, then an exception condition is raised: data exception as the invalid value for argument.
3. `input_text` shall not be the zero-length string. If it is, then an exception condition is raised: data exception as the invalid value for argument.

Formal Definition

1. Let IT be the value of the `input_text` parameter.
2. Let M be the value of the `model` parameter and let S be the value of the `settings` parameter.
3. Let L be the length of IT in the units given by `M.capacity_unit`. The determination of L is implementation-defined.
4. If L is greater than `M.input_capacity`, then an exception condition is raised: data exception as the input exceeds model capacity.
5. Let RT be the result of one inference of the model identified by M on IT, subject to S. The manner in which the model is identified and invoked is implementation-defined.
6. If no result was obtained because the model declined to produce one, then an exception condition is raised: model inference exception as the result withheld.
7. If the production of RT ended because a limit on result length was reached rather than because the model completed its result, then a warning condition is raised: model inference warning as the result truncated. RT is still the result of the invocation.
8. The result of the invocation of the function is RT.

NOTE: Step 7 exists because a truncated result and a short complete result are indistinguishable by inspection, so an invoker that is not informed of truncation has no means of detecting it.

Determinism

Reproducible on request (4.4.1).

5.2 MODEL_CLASSIFY

Purpose. Returns one label from a supplied set of labels for the input.

Kind. Scalar. **Status.** Conditional. **Model kind.** Classification or generation.

Form A, prompted labeling. A generation model is given the input and the label set and asked to pick one. This is MODEL_GENERATE plus §2.5 validation against the label set, and needs no separate entry beyond existing implementation.

Form B, trained classifier inference. A classification model is invoked. This is established machine learning work rather than anything new and it is the form that gives useful accuracy on a stable label set.

In-database training and inference exist in MADlib library however only as an analytical workload library. Its algorithms don't operate in the transactional deployments from which a scalar function in a query would ordinarily be called. So it establishes that in-database training is workable, not that a trained model is available where this function would be invoked. That distance between the two is the lifecycle problem stated below, not an argument against it.

The difficulty with form B is the lifecycle. A classifier is not a one-off call: it is a recurring pre-task, retrained daily, weekly, or monthly, and its output is only meaningful relative to the training run behind it. Standardizing the inference call while leaving production and refresh of the model entirely outside the standard gives users half of a workflow and no way to reason about staleness. Either the library also addresses training or refresh, at least to the extent of making the model version and training run identifiable through the model reference (§2.1), or form B should be deferred under §1.4(3) on the grounds that there is no invariant to hold an implementation to.

Determinism

Reproducible on request (4.4.1).

6. Embedding Functions

6.1 MODEL_EMBED

Purpose

Returns a vector representation of the input, produced by a named embedding model.

Kind. Scalar. **Status.** Recommended. **Model kind.** Embedding. **Input modality.** Character; binary where the model reference declares it.

Function Signature

```SQL
FUNCTION model_embed (
  input_text : CHARACTER STRING,
  model      : model_descriptor
) RETURNS VECTOR
```

Parameter Definitions

`input_text:` the value given to the model.
`model:` the descriptor of the model to be invoked.

Result

Returns VECTOR of M.result_dimension.

Argument Considerations

1. `model.model_kind` shall be `'EMBEDDING'`. If it is not, then an exception condition is raised: data exception as the invalid value for argument.
2. `model.result_dimension` shall be greater than 0. If it is not, then an exception condition is raised: data exception as the invalid value for argument.
3. `model.space_identity` shall not be the zero-length string. If it is, then an exception condition is raised: data exception as the invalid value for argument.

NOTE 1: Argument Consideration 3 requires a space identity for every invocation, because a stored vector whose space of origin is unknown cannot afterwards be established to be comparable with any other vector.

NOTE 2: This function does not compare vectors. It records which space a vector comes from, through `model.space_identity`, so that the `VECTOR` comparison can check it (§2.2).

Formal Definition

1. Let IT be the value of the `input_text` parameter and let M be the value of the `model` parameter.
2. Let L be the length of IT in the units given by `M.capacity_unit`. The determination of L is implementation-defined.
3. If L is greater than `M.input_capacity`, then an exception condition is raised: data exception as input exceeds model capacity.
4. Let V be the result of one inference of the model identified by M on IT. The manner in which the model is identified and invoked is implementation-defined.
5. If the number of dimensions is not equal to `M.result_dimension`, then an exception condition is raised: data exception as in invalid value for argument.
6. The result of the invocation of the function is V.

Determinism

Reproducible on request.

NOTE: An implementation serving the model locally at fixed numeric precision may be able to meet the Determined class.

7. Aggregate Functions

7.1 SUMMARIZE_COLLECTION

Purpose. Return a character string summarizing all values in a collection, for a collection whose combined size exceeds the input capacity of the model.

Function Signature

```
SQL
FUNCTION summarize_collection (
  input_collection : COLLECTION (CHARACTER STRING),
  model            : model_descriptor,
  settings         : inference_settings DEFAULT «default settings»
) RETURNS CHARACTER STRING
```

Parameter Definitions

`input_collection`: the values to be summarized.

`model`: the descriptor of the generation model to be invoked, per 4.2.1.

`settings`: the settings of each inference, per 4.2.2.

Result

Return a summary of all values in `input_collection`.

Argument Considerations

1. The cardinality of `input_collection` shall be greater than 0. If the cardinality is 0, then a no-data condition is raised.
2. `model.model_kind` shall be `'GENERATION'`. If it is not, then an exception condition is raised: data exception as in invalid value for argument.
3. An invocation shall succeed irrespective of the combined length of the values in `input_collection`. The combined length is not restricted by `model.input_capacity`. Because producing one summary of one value is `model_generate`; producing one summary of a collection larger than the model accepts in a single inference is not.

Formal Definition

1. Let IC be the value of the input_collection parameter, let M be the value of the model parameter, and let S be the value of the settings parameter.
2. Let C be the value of `M.input_capacity`.
3. Let W be a list whose elements are the values in IC. If IC is of an unordered collection type, the order of the elements of W is implementation-dependent.
4. Let I be an instruction to summarize, expressed as a character string. The content of I is implementation-defined.
5. If the cardinality of W is 1 and the combined length of I and the single element of W does not exceed C, then the result of the invocation of the function is `model_generate` (the concatenation of I and that element, M, S), and no further step is performed.
6. Let G be a list of lists of character strings, formed by taking the elements of W in order and appending each to the last element of G, except that a new element of G is started whenever appending would cause the combined length of I and the elements of that list to exceed C. The determination of length is implementation-defined, as in 5.1.
7. If any single element of W is such that the combined length of I and that element alone exceeds C, then an exception condition is raised: data exception as in input exceeds model capacity.
8. Let P be a list of character strings of the same cardinality as G, where, for all k, $1 \leq k \leq$ cardinality of G, P[k] is the result of `model_generate` (the concatenation of I and the elements of G[k], M, S).
9. If the cardinality of P is 1, then the result of the invocation of the function is P[1]. Otherwise, if the cardinality of P is equal to the cardinality of W, then an exception condition is raised: data exception: input exceeds model capacity. Otherwise, let W be P and repeat from step 6.

8. Ranking Functions

8.1 Rank fusion (proposed addition to SQL/Foundation)

Purpose. Compute a single ranking from two or more rankings by reciprocal rank fusion.

This clause defines the result of the rank fusion construct proposed in §1.2. The syntax of the construct is not defined here. The signature below describes its inputs and its result. This function invokes no model and its result is fully determined by its arguments, so an implementation of it can be tested by comparison of values.

Function Signature

```
SQL
FUNCTION fuse_rankings (
  rankings : LIST (ranking_rec),
  k        : NUMBER DEFAULT 60
) RETURNS LIST (fuse_rankings_ret)
```

```SQL
RECORD ranking_rec (
  keys   : LIST (CHARACTER STRING),
  weight : REAL DEFAULT 1
)

RECORD fuse_rankings_ret (
  key   : CHARACTER STRING,
  score : REAL,
  rank  : INTEGER
)
```

Parameter Definitions

`rankings:` the rankings to be fused. Within each `ranking_rec`, keys is in order of decreasing preference and weight is the relative contribution of that ranking to the result.

`k:` a constant that reduces the influence of the highest positions.

Argument Considerations

1. The cardinality of rankings shall be greater than or equal to 2. If it is not, then an exception condition is raised: data exception as in invalid value for argument.
2. k shall be greater than 0. If it is not, then an exception condition is raised: data exception as in invalid value for argument.
3. Within one `ranking_rec`, the values of keys shall be distinct. If they are not, then an exception condition is raised: data exception as in invalid value for argument.
4. A key is not required to appear in every ranking.
5. `weight` shall be greater than 0 (zero) in every `ranking_rec`. If it is not, then an exception condition is raised: data exception as in invalid value for argument.

Formal Definition

1. Let R be the value of the rankings parameter, let N be the cardinality of R, and let K be the value of k.
2. Let U be the set of keys that appear in the keys of at least one element of R.
3. For each key d in U, let A(d) be the set of integers i, $1 \leq i \leq N$, such that d appears in R[i].keys, and for each such i let r(i, d) be the position of d in R[i].keys, the first position being 1.
4. For each key d in U, let SCORE(d) be the value computed by Equation 1, "Reciprocal rank fusion score".
    a. $SCORE(d) = \sum_{i \text{ in } A(d)}^{1} \frac{R[i].weight}{K+r(i,d)}$

5. Let L be a list of records of type `fuse_rankings_ret`, one for each key d in U, with key set to d and score set to SCORE(d), ordered by score descending.
6. For all j, 1 ≤ j ≤ cardinality of L, `L[j].rank` is set to j, except that if `L[j].score` is equal to `L[j-1].score` then `L[j].rank` is set to `L[j-1].rank`.
7. The relative order within L of records having equal score is implementation-dependent.
8. The result of the invocation of the function is L.

Conclusion

We have presented here an outline of how a function library of "AI"-related functions could be specified, with the definition of important concepts, inclusion criteria, and various example functions. Additionally, several desirable extensions to database language standards (SQL and GQL) have been laid out.

If WG3 were to agree to this general direction, we are prepared to contribute substantially to the further development of this material.

In the short term, it would be important to agree on the timing and progression of the involved document projects so that further work could be planned.

Annex A. Mapping to existing implementations

Extending the survey in [W32-023] §§1.8–1.11. No endorsement of any implementation is intended and no statement is made as to fitness for purpose. Cells are function names and a modality or level note where relevant, not descriptions. Sources are product documentation; no hands-on testing is claimed.

This document	Snowflake	Databricks	BigQuery	EDB PG AI	«others»
`model_generate`	`AI_COMPLETE`	`ai_gen`, `ai_query`	`AI.GENERATE`	`aidb.decode_text`	«»
`model_embed`	`AI_EMBED`	«»	«»	`aidb.encode_text`	«»
`Vector comparison (§2.2)`	`AI_SIMILARITY`	`ai_similarity`	«»	pgvector `<=>, <->`	«»
`summarize_collection`	`AI_SUMMARIZE_AGG`	«»	`AI.AGG`	`aidb.summarize_text_aggregate`	«»

This document	Snowflake	Databricks	BigQuery	EDB PG AI	«others»
`fuse_rankings`	none	none	none	none	«»
`generate_boolean`	`AI_FILTER`	«»	`AI.IF`	«»	«»
`model_classify`	`AI_CLASSIFY`	`ai_classify`	`AI.CLASSIFY`	«»	«»
`parse_document`	`AI_PARSE_DOCUMENT`	`ai_parse_document`	«»	`aidb.parse_pdf`	«»
result structure honoured	«»	«»	«»	one provider	«»

Annex B. Function Examples

B.1 Sample Data

The table from [W32-023] §1.2, with a `review_id` column added. Lengths are in characters.

review_id	restaurant_name	city	comment	length
1	Pike Diner	Seattle	Great chowder and the staff remembered my name.	47
2	Pike Diner	Seattle	Fast service, fair prices, busy at noon.	40
3	Pike Diner	Seattle	The fish tacos were fresh and very generous.	44
4	Pike Diner	Seattle	Loud music, but great coffee.	29
5	Harbor Noodle	Seattle	Noodles were cold and the wait was long.	40

B.2 Model References

Assume three model references with these descriptor values (§4.2.1). The capacity of gen_small is small on purpose, so that the grouping in B.5 is visible. `embed_a` and `embed_b` have the same dimension on purpose (§2.2).

property	small_gen_model	embed_a	embed_b
model_kind	GENERATION	EMBEDDING	EMBEDDING
input_capacity	150	512	512
capacity_unit	CHARACTERS	CHARACTERS	CHARACTERS
result_dimension	0	768	768
space_identity	''	'embed_a/1'	'embed_b/1'
reproducible	True	True	True

B.3 model_generate (§5.1)

```
SQL
SELECT review_id,
       model_generate('Is this review about food or service? Answer in one word: '
                      || comment, small_gen_model) AS topic
FROM restaurant_reviews
WHERE restaurant_name = 'Pike Diner';

 review_id |  topic
-----------+---------
         1 | food
         2 | service
         3 | food
         4 | food
(4 rows)
```

According to the model references in §B.2 the longest input is 58 + 47 = 105 characters, which is within the capacity of 150.

With capacity constraint;

```
SQL
SELECT model_generate(comment || comment || comment || comment, small_gen_model)
FROM restaurant_reviews
WHERE review_id = 1;

ERROR:  data exception: input exceeds model capacity
DETAIL:  Input length is 188 CHARACTERS; input_capacity of small_gen_model is 150.
```

Run twice, both executions returns same because of status is reproducible under same settings :

```
SQL
SELECT model_generate('Describe this review: ' || comment, small_gen_model,
                      ROW(0, 0, 7, ''))       -- randomness 0, seed 7
FROM restaurant_reviews
WHERE review_id = 1;

                          model_generate
-----------------------------------------------------------------
 Guests say the chowder is great and the staff remember regulars.
(1 row)
```

B.4 summarize_collection (§7.1)

The result shows one row per group. The three inferences for Pike Diner, described step by step below this block, are not visible in the result.

```
SQL
SELECT restaurant_name,
       summarize_collection(comment, small_gen_model) AS summary
FROM restaurant_reviews
WHERE city = 'Seattle'
GROUP BY restaurant_name
ORDER BY restaurant_name;

 restaurant_name |                          summary
-----------------+-------------------------------------------------------
 Harbor Noodle   | Cold noodles and a long wait.
 Pike Diner      | Good food and friendly, fast staff, but it can be loud.
(2 rows)
```

Order matters with the result because with W in the order 1, 3, 2, 4, the same query could return:

```
SQL
 restaurant_name |                          summary
-----------------+-------------------------------------------------------------
 Harbor Noodle   | Cold noodles and a long wait.
 Pike Diner      | Fresh food and quick, friendly staff; the music is loud.
(2 rows)
```

B.5 model_embed and vector comparison (§6.2)

```
SQL
ALTER TABLE restaurant_reviews ADD COLUMN comment_vec VECTOR(768);
UPDATE restaurant_reviews SET comment_vec = model_embed(comment, embed_a);

ALTER TABLE
UPDATE 5
```

Distances in below are examples and lower distance means closer to the input. Additionally `<=>` is the pgvector cosine distance operator, used here only to illustrate.

```
SQL
SELECT review_id,
       comment_vec <=> model_embed('great fresh food', embed_a) AS distance
FROM restaurant_reviews
ORDER BY distance
LIMIT 4;

 review_id | distance
-----------+----------
         3 |    0.182
         1 |    0.264
         5 |    0.407
         2 |    0.519
(4 rows)
```

§2.2 prevents from a silent failure from the case of a query text is embedded with a different model(embed_b):

```SQL
SELECT review_id,
       comment_vec <=> model_embed('great fresh food', embed_b) AS distance
FROM restaurant_reviews
ORDER BY distance
LIMIT 4;
```

Both vectors have 768 dimensions, so today this runs and returns an ordering, but the ordering means nothing. Under §2.2 requirement 2, the comparison raises *data exception: incomparable embedding spaces* instead, because `comment_vec` comes from `embed_a` and the query vector comes from `embed_b`. This check is possible only because `model_embed` records the space identity.

```SQL
ERROR:  data exception: incomparable embedding spaces
DETAIL:  Left operand is from space 'embed_a'; right operand is from space
'embed_b'.
```

B.6 Rank Fusion (§8.1)

A keyword search for the same text, "great fresh food", matches "great" in reviews 1 and 4 and "fresh" in review 3, giving the ranking [1, 4, 3]. The inputs to rank fusion are then:

- R[1], vector ranking (B.5): keys '3', '1', '5', '2', weight 1
- R[2], keyword ranking: keys '1', '4', '3', weight 1
- k = 60

key	Position in R[1]	Position in R[2]	SCORE(Eq in §8.1)	rank
1	2	1	1/62 + 1/61 = 0.032522	1
3	1	3	1/61 + 1/63 = 0.032266	2
4	—	2	1/62 = 0.016129	3
5	3	—	1/63 = 0.015873	4
2	4	—	1/64 = 0.015625	5

Review 1 is only second in the vector ranking, but it is first overall, because it ranks high in both. Review 4 does not appear in the vector ranking at all (§8.1, Argument Consideration 4), but it still ranks above reviews 5 and 2. Unlike B.3 to B.5, these values are exact. No model is involved, so a test can compare them directly.

```
SQL
 key |  score   | rank
-----+----------+------
 1   | 0.032522 |    1
 3   | 0.032266 |    2
 4   | 0.016129 |    3
 5   | 0.015873 |    4
 2   | 0.015625 |    5
(5 rows)
```

— end of the paper —